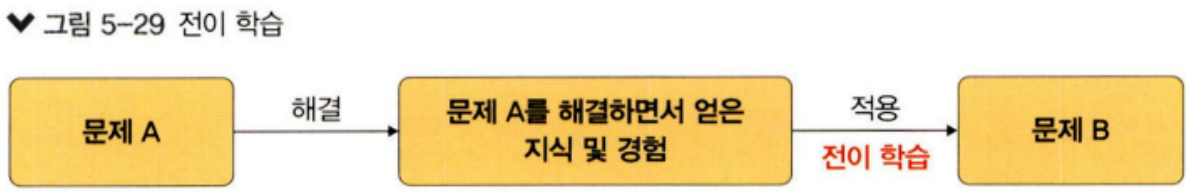
딥러닝 파이토치 교과서 5장 - Part 2

5장- 합성곱 신경망 I

- 5.3 전이 학습(p. 200 ~ p. 228)
- 5.4 설명 가능한 CNN(p. 229 ~ 239)
- 5.5 그래프 합성곱 네트워크(p. 240 ~ 244)

5.3 전이 학습



전이학습: **이미지넷(ImageNet)**처럼 아주 큰 데이터셋을 썻 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것

- 아주 큰 데이터셋을 사용하여 우리가 해결하려는 과제에 맞게 보정해서 사용함
- 큰 데이터셋을 확보하기 위해 많은 돈과 시간이 필요하다는 어려움에 대한 해결책
- 사전 훈련된 모델(네트워크): 아주 큰 데이터셋을 사용하여 훈련된 모델
- 비교적 적은 수의 데이터를 가지고 원하는 과제 해결 가능

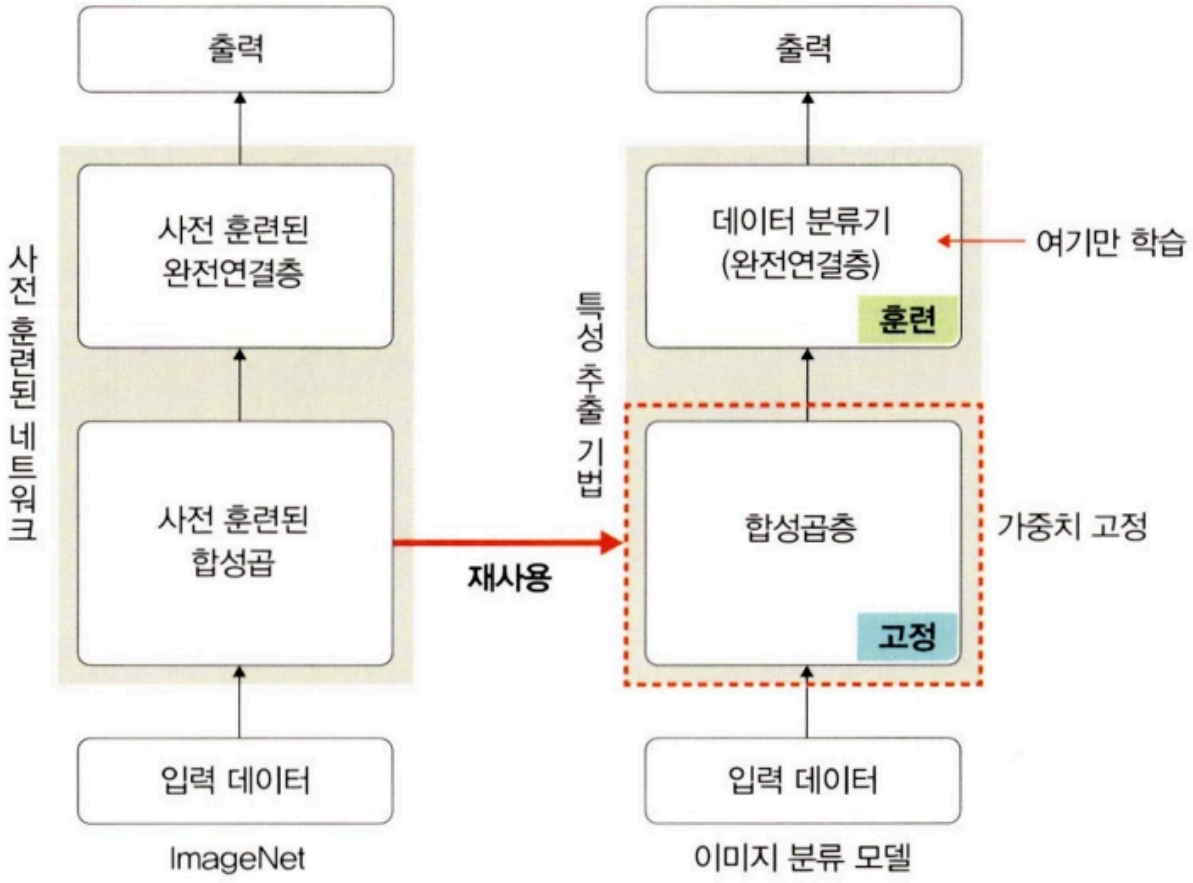
▼ +) ImageNet

  - 영상 인식 기술의 성능을 평가하는 주된 이미지 데이터셋
  - 2만개 이상의 클래스와 약 1400만장의 이미지로 구성됨

- 전이 학습을 위한 방법으로 **특성 추출 기법**과 **미세 조정 기법**이 있음

5.3.1 특성 추출 기법

그림 5-30 특성 추출 기법



특성 추출: ImageNet 데이터셋으로 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만듦

- 학습할 때는 **마지막 완전연결층**(이미지의 카테고리를 결정하는 부분)만 학습하고 나머지 계층들은 학습되지 않도록 함
- 이미지 분류를 위해 두 부분으로 구성됨

    - 합성곱층: 합성곱층과 풀링층으로 구성(사전 훈련된 네트워크)
    - 데이터 분류기(완전연결층): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류하는 부분
- 사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과시키고, 그 출력(추출된 특성)을 데이터 분류기에서 훈련
- 사용 가능한 이미지 분류 모델
    - Xception
    - Inception V3
    - ResNet50
    - VGG16
    - VGG19
    - MobileNet

5-12 라이브러리 호출

- OpenCV: 오픈 소스 컴퓨터 비전 라이브러리

```
import os
import time
import copy
import glob
import cv2
import shutil

import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim
```

```
import matplotlib.pyplot as plt
```

```
from google.colab import files # 데이터 불러오기
file_uploaded=files.upload()   # 데이터 불러오기: chap05/data/c
atndog.zip 파일 선택

!unzip catanddog.zip -d catanddog/    #catanddog 폴더 만들어 압
축 풀기
```

5-13 이미지 데이터 전처리 방법 정의

- torchvision.transfrom: 이미지 데이터를 변환하여 모델(네트워크)의 입력으로 사용할 수 있게 변환해 줌
  - Resize: 이미지의 크기를 조정 >> 256x256 크기로
  - RandomResizedCrop(224): 이미지를 랜덤한 크기 및 비율로 자름
    - Resize: 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정
    - RandomResizedCrop: 데이터 확장 용도로 사용됨
  - RandomHorizontalFlip: 이미지를 랜덤하게 수평으로 뒤집음
  - ToTensor: 이미지 데이터를 텐서로 변환함
- datasets.ImageFolder: 데이터로더가 데이터를 불러올 대상(혹은 경로)과 방법(혹은 전처리를 정의)
  - data_path: 불러올 데이터가 위치한 경로
  - transform: 이미지 데이터에 대한 전처리
- Dataloader: 데이터를 불러오는 부분
  - ImageFolder(train_dataset)을 데이터로더에 할당
  - 한 번에 불러올 데이터양을 결정하는 batch_size 지정
  - 데이터를 무작위로 섞을(shuffle) 것인지도 설정
  - 사용하는 파라미터

- train_dataset: 데이터셋 지정
- batch_size: 한 번에 불러올 데이터양을 결정하는 배치 크기를 설정
- num_workers: 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지 설정
  - 데이터 로딩 속도에 영향
  - 너무 많은 하위 프로세스를 설정하게 되면 오류가 발생하거나 메모리 부족 현상이 발생할 수 있음
- shuffle: 데이터를 무작위로 섞을지 지정
  - shuffle=True: 데이터를 무작위로 섞어서 랜덤으로 불러옴

```
data_path = 'catanddog/train/' # 이미지 데이터가 위치한 경로 지정

# 여러 변환(transformations)을 함께 묶어 순차적으로 적용
transform = transforms.Compose(
                [
                    transforms.Resize([256, 256]),
                    transforms.RandomResizedCrop(224),
                    transforms.RandomHorizontalFlip(),
                    transforms.ToTensor(),
                ])

# 지정된 경로에서 이미지를 불러오고 앞서 정의한 변환을 적용하여 데이터
셋을 생성
# ImageFolder: 주어진 디렉토리 구조에 따라 이미지를 자동으로 라벨링
# torchvision.transfrom: 이미지 데이터를 변환하여 모델(네트워크)의
입력으로 사용할 수 있게 변환해 줌
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)

# 데이터셋에서 배치(batch) 단위로 데이터를 로드하기 위한 데이터 로더를
생성
```

```
# batch_size=32: 한 번에 32개의 이미지를 로드
# num_workers=8: 데이터 로딩을 위해 8개의 병렬 작업자(threads)를 사용
# shuffle=True: 데이터를 무작위로 섞어서 모델이 데이터의 순서에 의존하지 않도록 함
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))
```

5-14 학습에 사용될 이미지 출력

- iter(): 전달된 데이터의 반복자를 꺼내 반환
- next(): 그 반복자가 다음에 출력해야 할 요소를 반환
  - train_loader에서 samples와 labels의 값을 순차적으로 꺼내 저장
  - train_loader에서 데이터를 하나씩 꺼내 오겠다는 의미
- n. transpose: 행과 열을 바꿈으로서 행렬의 차원을 바꿔줌
  - 행렬의 차원을 변경하는 이유: 행렬의 내적 연산 때문
  - A와 B를 내적하려면 A 행렬의 행과 B 행렬의 열 수가 같아야 함(행렬곱)
  - A와 B의 행과 열의 수가 달라 내적이 불가능할 경우 np.transpose()나 np.reshape() 등으로 차원을 조정해야 함
- np.transpose(samples[i].numpy(), (1,2,0))
  - samples[i].numpy()는 PyTorch 텐서를 numpy 배열로 변경한 것
  - 1은 원래 배열의 두 번째 차원(높이)이 첫 번째 위치로 이동한다는 것을 의미
  - 2는 원래 배열의 세 번째 차원(너비)이 두 번째 위치로 이동한다는 것을 의미
  - 0은 원래 배열의 첫 번째 차원(채널)이 세 번째 위치로 이동한다는 것을 의미

- 결과적으로 (채널, 높이, 너비) 형식의 배열이 (높이, 너비, 채널) 형식으로 변경됨 ➡️ matplotlib을 사용하여 이미지를 시각화할 때 올바른 형식으로 표시
- PyTorch는 이미지를 (채널, 높이, 너비) 형식으로 저장하지만, matplotlib은 (높이, 너비, 채널) 형식을 사용

```
import numpy as np
samples, labels = next(iter(train_loader))
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24):
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보(클래스)
를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

5-15 사전 훈련된 모델 내려받기

- 사전 훈련된 ResNet18 모델을 내려받음
- ResNet18
  - 50개의 계층으로 구성된 합성곱 신경망
  - ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망으로, 학습에 사용되도록 사전 훈련된 모델을 제공하고 있음
  - 입력 제약이 매우 크고, 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있다는 단점이 있음
- resnet18 = models.resnet18(pretrained=True)
  - resnet18 = models.resnet18(): 파이토치에서 무작위의 가중치로 모델을 구성
  - pretrained=True: 사전 학습된 모델(사전 학습된 모델의 가주치 값)을 사용할 수 있도록 하는 파라미터, 사전 학습된 가중치를 사용하겠다는 의미

```
resnet18 = models.resnet18(pretrained=True) # 사전 학습된 가중치 사용
```

5-16 사전 훈련된 모델의 파라미터 학습 유무 지정

- param.requires_grad = False
  - 역전파 중 파라미터들에 대한 변화를 계산할 필요가 없음을 의미
  - 모델의 일부(합성곱층, 풀링층)를 고정하고 나머지를 학습하고자 할 때 requires_grad = False로 설정
- model.parameters(): 모델에 있는 모든 매개변수를 반환하는 함수
- param.requires_grad = False: 순회 중인 매개변수의 requires_grad 속성을 False로 설정
  - 해당 매개변수에 대한 그래디언트 계산을 비활성화하는 것으로, 모델을 훈련시킬 때 이 매개변수를 고정시키고 업데이트하지 않겠다는 의미
  - 주로 전이 학습(Transfer Learning)에서 특성 추출기로 모델을 사용할 때 필요함
  - 전이학습에서는 사전에 학습된 모델의 대부분의 파라미터를 재사용하며, 일부만 새로운 데이터셋에 맞게 조정함
  - requires_grad = False를 설정함으로써 학습해야 할 파라미터의 수가 줄어들어 학습 속도가 향상
  - 전이학습의 경우 초기 계층의 파라미터를 고정(requires_grad = False)하고 마지막 계층만을 학습시키는 것이 일반적

```
# model은 PyTorch 모델을 의미
# feature_extracting은 특성 추출을 할 것인지 여부를 나타내는 불리언 값
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting: # feature_extracting 매개변수가 True일 경우에만 실행
        for param in model.parameters(): # 모델의 모든 매개변수
```

```
를 순회하는 반복문
            param.requires_grad = False

set_parameter_requires_grad(resnet18)
```

5-17 ResNet18에 완전연결층 추가

- 내려받은 ResNet의 마지막 부분에 완전연결층 추가
  - 입력 특성의 크기가 512인 이유: ResNet-18 모델에서는 마지막 convolutional layer를 거친 후, 특성 맵을 평탄화(flatten)하여 fully connected layer로 넘기기 전의 특성 벡터의 크기가 512가 되기 때문
- 추가된 완전연결층은 개와 고양이 클래스를 분류하는 용도로 사용됨

```
# .fc: ResNet-18 모델의 fully connected layer(완전 연결층)
# nn.Linear(512, 2): 입력 특성의 크기가 512이고 출력 특성의 크기가
2인 선형 변환
# 2는 클래스가 두 개라는 의미
resnet18.fc = nn.Linear(512, 2)
```

5-18 모델의 파라미터 값 확인

- resnet18.named_parameters(): 모델의 파라미터 이름과 값을 순회할 수 있는 반복자를 제공
  - 모델의 각 파라미터에 접근 가능
- if param.requires_grad: 조건문을 사용하여 그래디언트 계산이 활성화된 파라미터만 출력
  - 학습 과정에서 업데이트될 파라미터를 확인하는 데 유용함

```
# model.named_parameters()는 모델에 접근하여 파라미터 값들을 가져올
때 사용
for name, param in resnet18.named_parameters():
```

```
    if param.requires_grad:
        print(name, param.data)
```

5-19 모델 객체 생성 및 손실 함수 정의

```
model = models.resnet18(pretrained = True) # 사전 학습된 ResNet
18 모델을 불러옴


for param in model.parameters():
    # 모델의 모든 파라미터에 대한 그래디언트 계산을 비활성화(전이 학습
시 기존 가중치를 고정하기 위함)
    param.requires_grad = False

# 모델의 마지막 Fully Connected(FC) 층을 새로운 작업에 맞게 교체
# torch.nn.Linear: 선형 변환을 적용하는 층
# param.requires_grad = True: 해당 층의 파라미터만 학습되도록 함
model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True

# Adam 최적화 알고리즘(학습률을 자동으로 조정)을 사용하여 새로운 FC 층
의 파라미터를 최적화
optimizer = torch.optim.Adam(model.fc.parameters())
# 분류 문제에 널리 사용되는 손실 함수
# 모델의 출력과 실제 레이블 간의 차이를 계산하여 모델이 더 정확한 예측
을 하도록 학습을 유도함
cost = torch.nn.CrossEntropyLoss()
print(model)
```

5-20 모델 학습을 위한 함수 생성

```
def train_model(model, dataloaders, criterion, optimizer, dev
ice, num_epochs=13, is_train=True):
    since = time.time() # 컴퓨터의 현재 시각을 구하는 함수

    # 각 에포크마다의 정확도와 손실 값을 저장할 리스트를 초기화
    acc_history = []
    loss_history = []
    # 지금까지의 최고 정확도를 저장할 변수를 초기화
    best_acc = 0.0

    for epoch in range(num_epochs): # 전체 에포크 수(13)만큼 반
복

        # 현재 에포크 번호를 출력
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        # 현재 에포크에서의 누적 손실 값과 정확히 분류된 데이터 수를
저장할 변수 초기화
        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: # 데이터로더에 전달된
데이터만큼 반복(배치 단위로 데이터 불러옴)
            inputs = inputs.to(device)
            labels = labels.to(device)

            # 모델을 현재 학습 장치로 이동시킴
            model.to(device)
            optimizer.zero_grad() # 기울기를 0으로 설정(최적화 알
고리즘의 기울기를 초기화)
            outputs = model(inputs) # 순전파 학습(모델을 통해 입
력 데이터에 대한 예측을 수행)
            loss = criterion(outputs, labels) # 손실 함수를 사
용하여 예측과 실제 레이블 간의 손실을 계산
```

```
            _, preds = torch.max(outputs, 1) # 예측 결과 중 가
장 높은 값을 가진 클래스를 선택
            loss.backward() # 역전파 학습(손실에 대한 기울기를 계
산)
            optimizer.step() # 최적화 알고리즘을 사용하여 모델의
가중치를 업데이트

            # 현재 배치에서의 손실과 정확도를 누적함
            # 출력 결과와 레이블의 오차를 계산한 결과를 누적하여 저장
            running_loss += loss.item() * inputs.size(0)
            # 출력 결과와 레이블이 동일한지 확인한 결과를 누적하여 저
장
            running_corrects += torch.sum(preds == labels.dat
a)

        # 전체 데이터셋에 대한 평균 손실과 정확도를 계산
        epoch_loss = running_loss / len(dataloaders.dataset)
# 평균 오차 계산
        epoch_acc = running_corrects.double() / len(dataloade
rs.dataset) # 평균 정확도 계산

        # 현재 에포크의 손실과 정확도를 출력
        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, e
poch_acc))

        # 현재 에포크의 정확도가 이전의 최고 정확도보다 높다면 업데이
트
        if epoch_acc > best_acc:
            best_acc = epoch_acc

        # 현재 에포크의 정확도와 손실을 기록
        acc_history.append(epoch_acc.item())
        loss_history.append(epoch_loss)
        # 현재 에포크의 모델 상태를 저장
        torch.save(model.state_dict(), os.path.join('catanddo
g/', '{0:0=2d}.pth'.format(epoch)))
```

```
        print()

    time_elapsed = time.time() - since # 학습에 걸린 총 시간을 계산

    # 학습 완료 메시지와 함께 최고 정확도를 출력
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:4f}'.format(best_acc))
    return acc_history, loss_history # 학습 과정에서 기록된 정확도와 손실의 이력을 반환
```

5-21 파라미터 학습 결과를 옵티마이저에 전달

- named_parameters(): 모델의 파라미터 이름과 파라미터 객체를 튜플 형태로 반환
- requires_grad == True: 학습 과정에서 그래디언트를 통해 업데이트 될 수 있음

```
params_to_update = [] # 학습 가능한 파라미터를 저장할 준비
for name,param in resnet18.named_parameters(): # 모델의 모든 파라미터와 그 이름을 순회
    if param.requires_grad == True: # 학습 가능한 파라미터(그래디언트 계산이 필요한 파라미터)를 식별

        # 학습 가능한 파라미터를 params_to_update 리스트에 추가하고 해당 파라미터의 이름을 출력
        # 어떤 파라미터가 학습 과정에 포함되는지 확인할 수 있음
        params_to_update.append(param)
        print("\t",name)

# Adam 옵티마이저 생성(학습 가능한 파라미터의 업데이트를 위해 사용됨)
# 학습률을 각 파라미터에 대해 독립적으로 조정하면서, 모맨텀을 사용하여 이전 그래디언트의 방향을 고려함
# params_to_update:  저장된 학습 가능한 파라미터들을 Adam 옵티마이저에 전달
```

```
# 학습 가능한 파라미터들이 파라미터들이 학습 과정에서 업데이트될 수 있도록 설정
optimizer = optim.Adam(params_to_update)
```

5-22 모델 학습

- 모델 훈련을 위해 전달되는 파라미터
  - 모델
  - 학습 데이터
  - 손실 함수
  - 옵티마이저
  - 디바이스(CPU or GPU)

```
# 디바이스 설정: GPU, CPU 중 선택
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# 손실 함수 설정:
criterion = nn.CrossEntropyLoss()
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion, optimizer, device)
```

5-23 테스트 데이터 호출 및 전처리

- 훈련 데이터로 학습 후 테스트 데이터를 통해 모델 정확도 측정
- 5-13과 유사

```
test_path = 'catanddog/test' # 데이터 경로 설정

# 여러 전처리 단계를 순차적으로 적용할 수 있도록 함
# 이미지를 224x224 크기로 리사이징, 중앙에서 224x224 크기로 크롭, 그리고 텐서로 변환하는 세 단계의 전처리를 설정
transform = transforms.Compose(
```

```
                    [
                        transforms.Resize(224),
                        transforms.CenterCrop(224),
                        transforms.ToTensor(),
                    ])

# 지정된 경로에서 이미지 데이터셋을 불러옴
# 주어진 디렉토리 구조에서 자동으로 레이블을 인식하여 데이터셋을 구성함
# transform 인자를 통해 데이터에 전처리를 적용
test_dataset = torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)

# 데이터를 배치 단위로 불러올 수 있는 로더를 생성
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=32, # 한 번에 32개의 이미지를 불러옴
    num_workers=1, # 데이터 로딩을 위해 사용할 프로세스 수를 지정
    shuffle=True # 데이터를 무작위로 섞어 모델이 데이터의 순서에 의
존하지 않도록 함
)

print(len(test_dataset)) # 테스트 데이터셋의 총 이미지 수 출력
```

5-24 테스트 데이터 평가 함수 생성

- 5-20과 유사

```
def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('catanddog/' + '*.pth')
```

```
    saved_models.sort()
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders:
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad():
                outputs = model(inputs)

            _, preds = torch.max(outputs.data, 1)
            preds[preds >= 0.5] = 1
            preds[preds < 0.5] = 0
            running_corrects += preds.eq(labels).int().sum()

        epoch_acc = running_corrects.double() / len(dataloade
rs.dataset)
        print('Acc: {:.4f}'.format(epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(tim
e_elapsed // 60, time_elapsed % 60))
```

```
    print('Best Acc: {:4f}'.format(best_acc))

    return acc_history
```

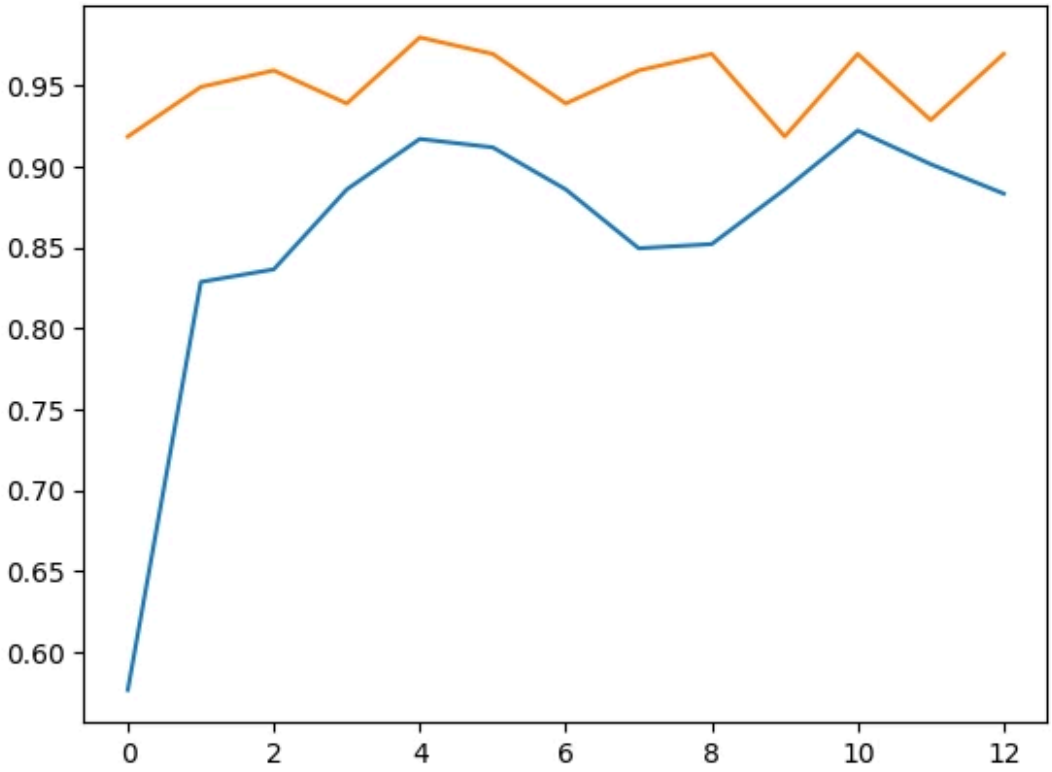
5-25 테스트 데이터를 평가 함수에 적용

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

5-26 훈련과 테스트 데이터의 정확도를 그래프로 확인

- 훈련과 테스트 데이터에 대해 에포크가 진행될 때마다 정확도를 출력한 결과
- 훈련과 테스트 데이터 모두 에포크가 진행될수록 정확도가 높아지면서 100%에 가까워짐

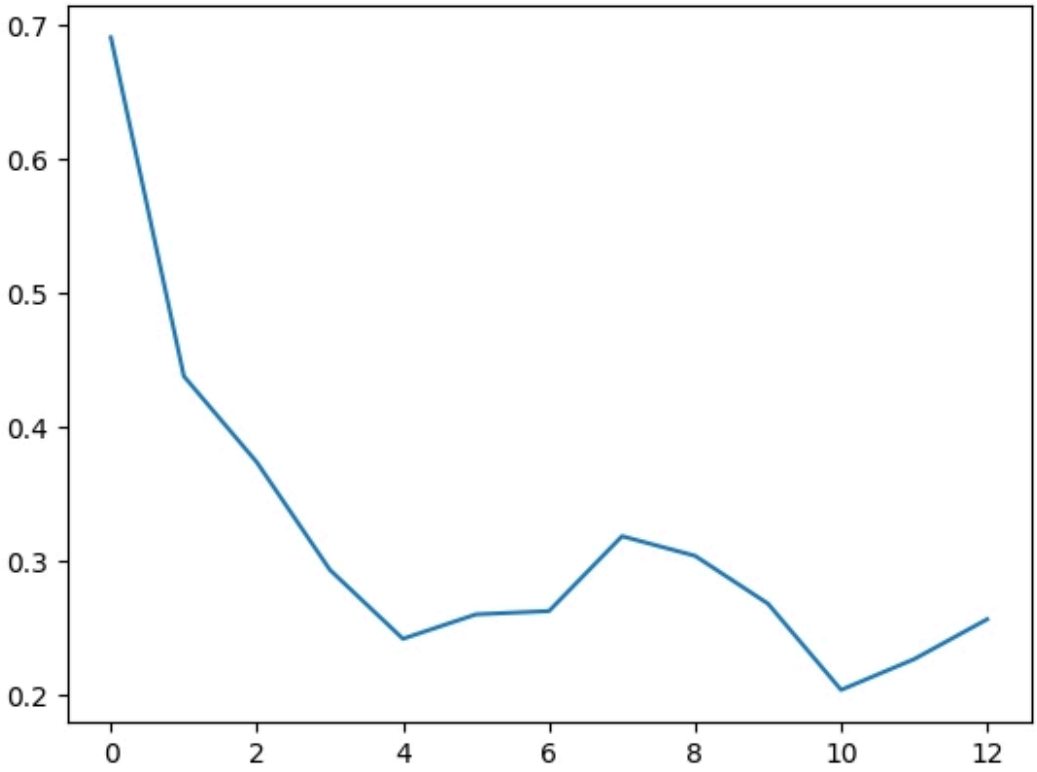
```
plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()
```



5-27 훈련 데이터의 오차에 대한 그래프 확인

- 훈련 데이터에 대해 에포크가 진행될 때마다 오차를 출력한 결과
- 에포크가 진행될수록 오차가 낮아지고 있기 때문에 학습이 잘 되었다고 볼 수 있음

```
plt.plot(train_loss_hist)
plt.show()
```



5-28 예측 이미지 출력을 위한 전처리 함수

- tensor.clone(): 기존 텐서의 내용을 복사한 텐서를 생성하겠다는 의미
- detach(): 기존 텐서에서 기울기가 전파되지 않은 텐서
    - tensor.clone().detach(): 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에 영향을 주지는 않겠다는 의미
- clip(): 입력 값이 주어진 범위를 벗어날 때, 입력 값을 특정 범위로 제한시키기 위해 사용
    - image.clip(0, 1): image 데이터를 0과 1사이의 값으로 제한

```
def im_convert(tensor):
    image=tensor.clone().detach().numpy()
    image=image.transpose(1,2,0) # 이미지 축 변환(파이토치 텐서는 (채널, 높이, 너비), 이미지는 (높이, 너비, 채널) 형태로 표현됨)
    image=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5))) # 정규화 해제, 원래의 값 범위로 되돌림
```

```
    image=image.clip(0,1) # 이미지 픽셀 값을 0과 1 사이로 제한
    return image
```

5-29 개와 고양이 예측 결과 출력

- add_subplot: 한 화면에 여러 개의 이미지를 담을 때 사용
- classes[preds[idx].item()]
  - preds[idx].item() 값이 classes로 정의된 0과 1중 어떤 값을 가지는지 판별
  - classes[preds[idx].item()] 값이 0이면 고양이, 1이면 개로 출력

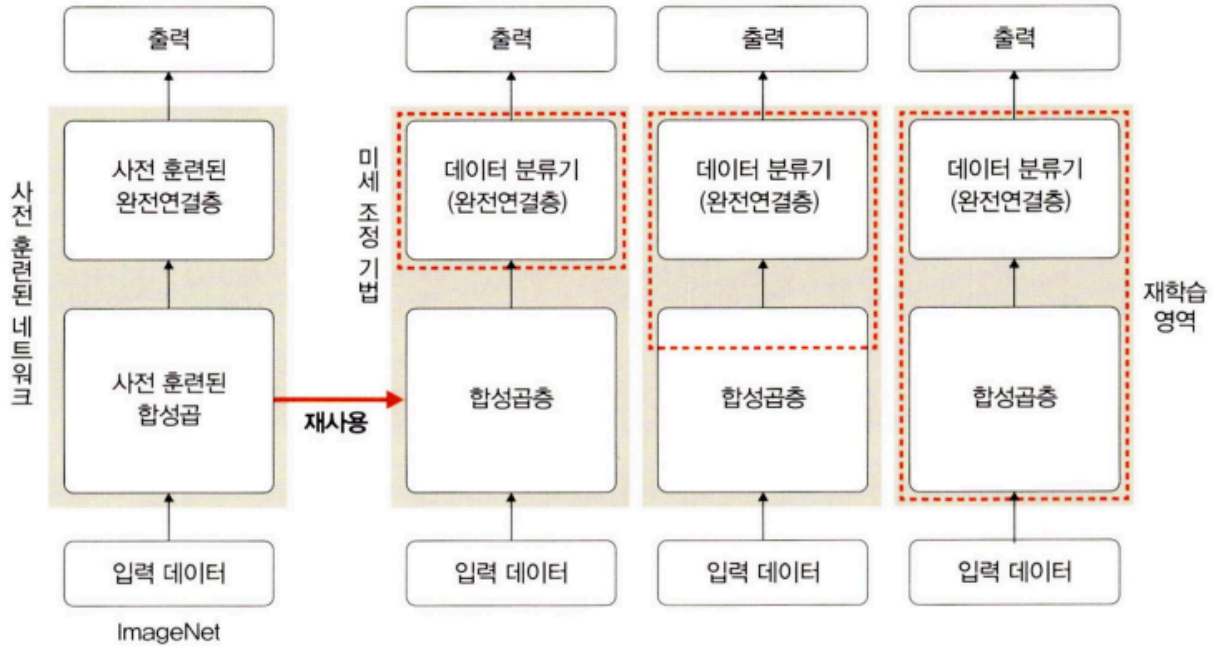
```
classes = {0:'cat', 1:'dog'} # 개와 고양이 두 개에 대한 레이블

dataiter=iter(test_loader)  # 테스트 데이터셋을 가져옴
images,labels=next(iter(test_loader)) # 테스트 데이터셋에서 이미지와 레이블을 분리하여 가져옴
output=model(images)
_,preds=torch.max(output,1)

fig=plt.figure(figsize=(25,4))
for idx in np.arange(20):
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])
    plt.imshow(im_convert(images[idx])) # 이미지 전처리를 위한 함수(5-28) 적용
    a.set_title(classes[labels[i].item()])
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]),str(classes[labels[idx].item()])),color=("green" if preds[idx]==labels[idx] else "red"))
plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

5.3.2 미세 조정 기법

그림 5-44 미세 조정 기법



미세 조정: 특성 추출 기법에서 더 나아가 **사전 훈련된 모델**과 **합성곱층**, **데이터 분류기**의 가중치를 업데이트하여 훈련시키는 방식

- 목표 특성을 잘 추출했다는 전제하에 좋은 성능을 낼 수 있음
- 특성이 잘못 추출되었다면 미세 조정 기법으로 새로운 이미지 데이터를 사용하여 네트워크의 가중치를 업데이트해서 특성을 다시 추출할 수 있음
- 사전 학습된 모델을 목적에 맞게 재학습시키거나, 학습된 가중치의 일부를 재학습시키는 것

- 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정
- 미세 조정 과정에서 많은 연산량이 필요하기 때문에 CPU보다 GPU 사용이 권장됨
- 파라미터 업데이트 과정에서 파라미터에 큰 변화를 주게 되면 과적합 문제가 발생할 수 있기 때문에 정교하고 미세한 파라미터 업데이트가 필요함

☑️ 데이터셋이 **크고** 사전 훈련된 모델과 유사성이 **작을 경우**➡️ **모델 전체**를 재학습

☑️ 데이터셋이 **크고** 사전 훈련된 모델과 유사성이 **클 경우** ➡️ **합성곱층의 뒷부분**(완전연결층과 가까운 부분)과 **데이터 분류기**를 학습시킴

5.4 설명 가능한 CNN

설명 가능한 CNN: 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시

- CNN 처리 과정을 시각화
- CNN을 구성하는 각 중간 계층부터 최종 분류까지 입력된 이미지에서 특성이 어떻게 추출되고 학습하는지를 시각적으로 설명할 수 있어야 결과에 대한 신뢰성을 얻을 수 있음
- CNN의 시각화 방법
  - 필터에 대한 시각화
  - 특성 맵에 대한 시각화

5.4.1 특성 맵 시각화

특성 맵: 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과

- 특정 입력 이미지에 대한 특성 맵을 시각화한다는 의미는 맵에서 **입력 특성을 감지하는 방법**을 이해할 수 있도록 돕는 것

5-30 필요한 라이브러리 호출

```
import matplotlib.pyplot as plt
from PIL import Image
```

```
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("cuda" if torch.cuda.is_available() els
e "cpu")
```

5-31 설명 가능한 네트워크 생성

- 13개의 합성곱층과 두 개의 완전연결층으로 구성된 네트워크
- 합성곱층과 완전연결층은 ReLU 활성화 함수를 사용
- log_softmax(): 신경망 말단의 결괏값들을 확률 개념으로 해석
    - 소프트맥스를 사용하지 않고 로그 소프트맥스를 사용하는 이유: 소프트맥스는 기울기 문제에 취약하기 때문

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2): # 클래스가 2개
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), # 기존의 데이터를 연산의 결괏
값으로 대체
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding = 1, bia
s=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
            nn.Conv2d(64, 128, kernel_size=3, padding = 1, bi
as=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(256, 512, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(512),
```

```
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(512, 512, bias=False),
            nn.Dropout(0.5),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(-1, 512) # 특성 맵에서 추출 후 분류기 거치기 전
```

```
에 평탄화 작업
        x = self.classifier(x)
        return F.log_softmax(x)
```

5-32 모델 객체화

- 앞서 생성된 모델을 model = XAI()로 객체화한 후 장치(CPU 혹은 GPU)에 할당함

```
model=XAI() # model 이라는 이름의 객체를 생성(인스턴스)
model.cpu() # 모델에 입력되는 이미지를 넘파이로 받아오는 부분때문에 C
PU를 사용하도록 지정
model.eval() # 테스트 데이터에 대한 모델 평가 용도로 사용
```

5-33 특성 맵을 확인하기 위한 클래스 정의

- 특성 맵은 합성곱층을 입력 이미지와 필터를 연산하여 얻은 결과 ➡️ 합성곱층에서 입력과 출력을 알 수 있다면 특성 맵에 대한 값들을 확인할 수 있음
- 5-32의 출력 결과인 Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)에 대한 특성 맵을 확인하기 위한 클래스를 먼저 정의
- 파이토치는 매 계층마다 print문을 사용하지 않더라도 hook 기능을 사용하여 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있음
    - register_forward_hook의 목적: 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져오는 것

```
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(se
lf.hook_fn)

    def hook_fn(self, module, input, output):
        output = output
        #self.features = output.to(device).detach().numpy()
```

```
        self.features = output.detach().numpy()
    def remove(self):
        self.hook.remove()
```

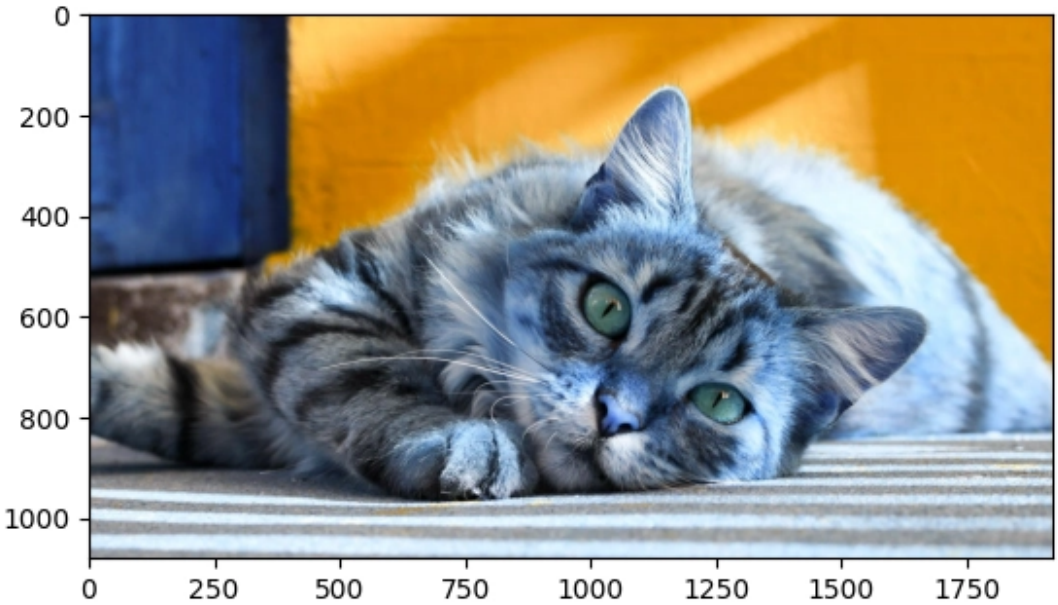
5-34 이미지 호출

- cv2.resize: 이미지 크기 변겨
  - img: 변경할 이미지 파일
  - (100, 100): 변경될 이미지 크기를 (너비, 높이)로 지정
  - interpolation: 보간법(추정해야 하는 픽셀에 대해)
- unsqueeze(): 1차원 데이터를 생성하는 함수
  - 이미지 데이터를 텐서로 변환 후 1차원으로 변경

```
from google.colab import files # 데이터 불러오기
file_uploaded=files.upload()   # chap05/data/cat.jpg 데이터 불
러오기

img=cv2.imread("cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LIN
EAR)
img = ToTensor()(img).unsqueeze(0)

print(img.shape)
```



5-35 (0): Conv2d 특성 맵 확인

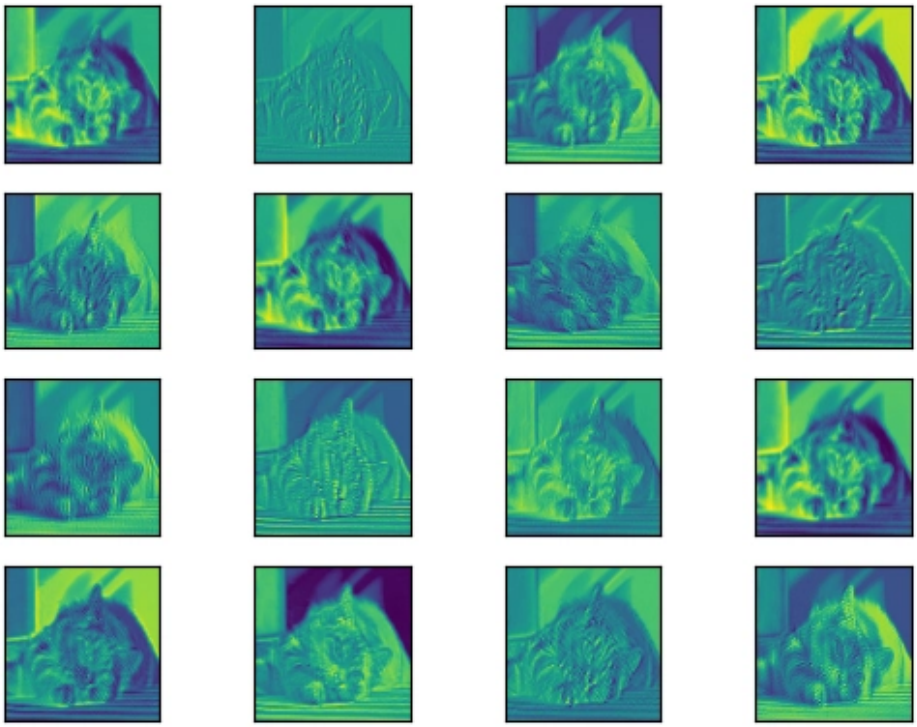
```
result = LayerActivations(model.features, 0) # 0번째 Conv2d 특성 맵 확인
model(img)
activations = result.features
```

5-36 특성 맵 확인

- 0번째 계층: 입력층과 가까운 계층으로 입력 이미지의 형태가 많이 유지됨

```
fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
```

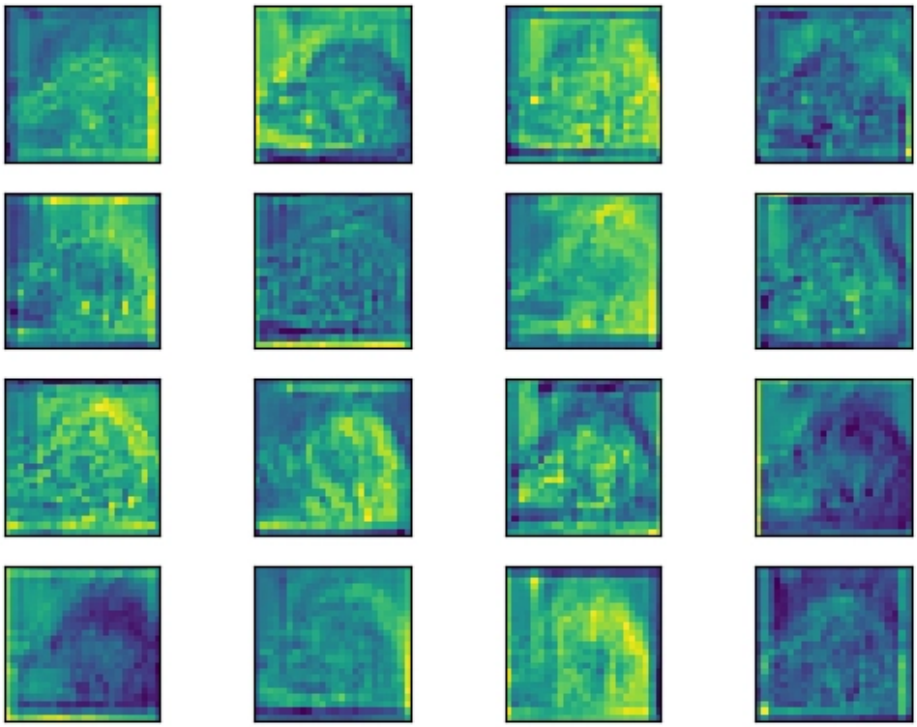
```
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```



5-38 특성 맵 확인

```
fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=
0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
```

```
        axis.imshow(activations[0][row*10+column])
plt.show()
```



5-39 40번째 계층에 대한 특성 맵

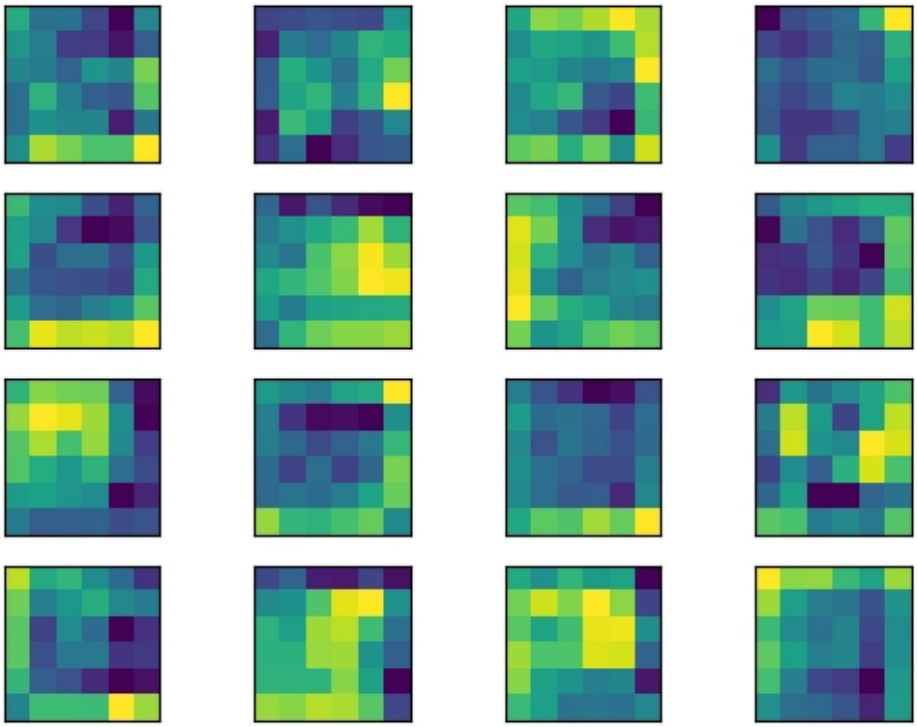
```
result = LayerActivations(model.features, 40) # 40번째 Conv2d
특성 맵 확인

model(img)
activations = result.features
```

5-40 특성 맵 확인

- 출력층에 가까워질수록 원래 형태를 잃어가고 이미지 특징들만 전달됨

```
fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=
0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```



5.5 그래프 합성곱 네트워크

5.5.1 그래프란

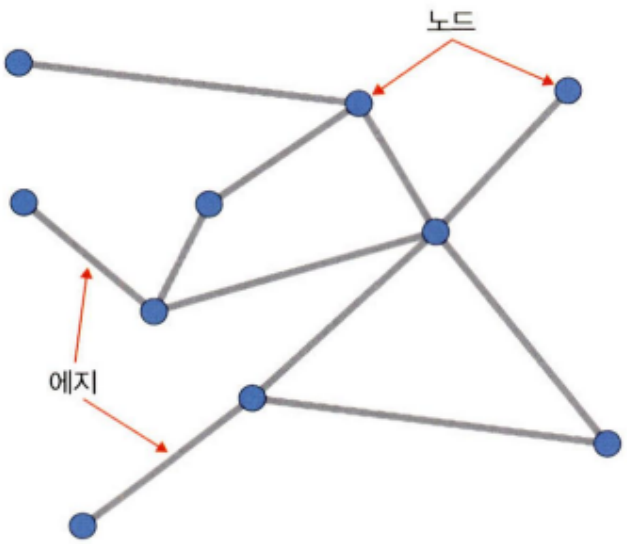


그림 5-50 그래프

그래프: 방향성이 있거나(directed) 없는(undirected) 에지로 연결된 노드의 집합

- 노드와 에지: 일반적으로 풀고자 하는 문제에 대한 전문가 지식이나 직관 등으로 구성됨
  - **노드**: 위 그림에서 파란색 원 ➡️ **원소**
  - **에지**: 두 노드를 연결한 선 ➡️ **결합 방법**

5.5.2 그래프 신경망

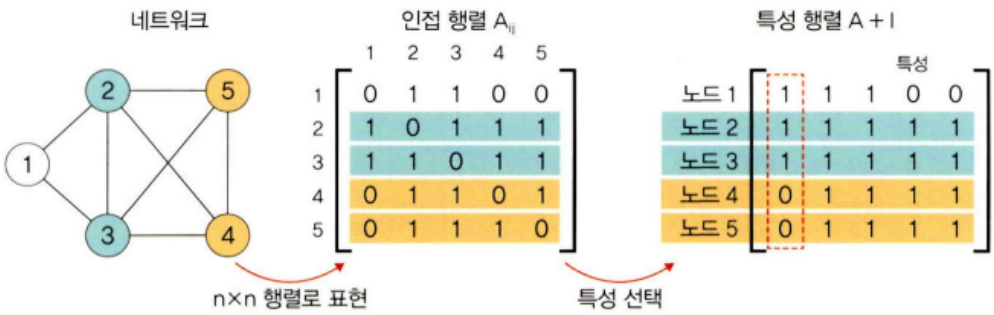


그림 5-51 특성 행렬[5]

그래프 신경망(GNN): 그래프 구조에서 사용하는 신경망, 그래프 데이터에 대한 표현은 두 단계로 이루어짐

1️⃣ **인접 행렬**(adjacency matrix)

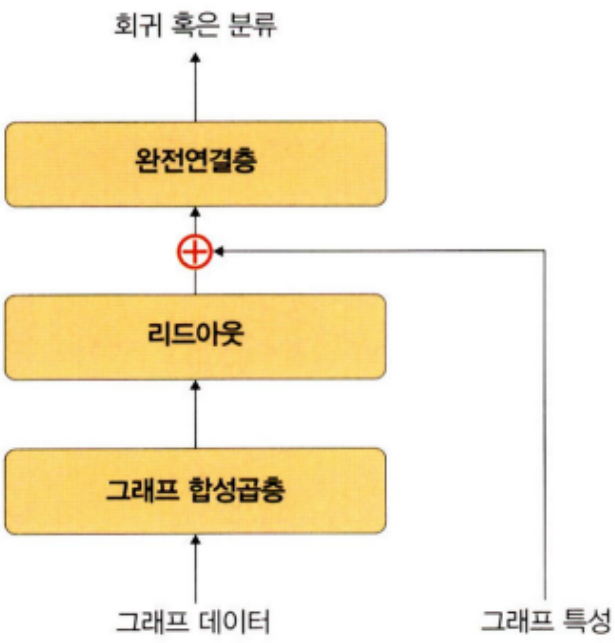
- 노드 n개를 nxn 행렬로 표현
- 인접 행렬 내의 값은 'Ay는 i와 j의 관련성 여부'를 만족하는 값으로 채워줌

2️⃣ **특성 행렬**(feature matrix)

- 인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬을 적용
- 각 입력 데이터에 이용할 특성을 선택
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미함
    - 첫 번째 행은 첫 번째 노드의 특성 값
- 이제 노드 1, 2, 3, 4에 대한 특성을 한눈에 파악하기 쉽게 표현됨

5.5.3 그래프 합성곱 네트워크

그림 5-52 그래프 합성곱 네트워크



그래프 합성곱 네트워크(GCN): 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

- **리드아웃**: 특성 행렬을 하나의 벡터로 변환하는 함수
  - 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성함
- **그래프 합성곱층**: 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있도록 함
- GCN의 활용
  - SNS에서 관계 네트워크
  - 학술 연구에서 인용 네트워크
  - 3D Mesh